Sudoku Solver – Backtracking Algorithm Implementation

Amol Vyas, Akshat Arya, Dr. Krishnappa H K

Computer Science Department, RV College of Engineering

Amolvyas.cs23@rvce.edu.in
Akshatarya.cs23@rvce.edu.in
krishnappahk.cs23@rvce.edu.in

***Abstract*— Sudoku, a popular logic-based puzzle, challenges players to fill a 9x9 grid with digits from 1 to 9, adhering to specific constraints. This project presents the development of a Sudoku solver using C++, demonstrating effective algorithm design and implementation. The program leverages backtracking, a widely-used algorithmic approach for constraint satisfaction problems, to efficiently explore potential solutions. In addition to the core solver, the project includes a graphical user interface (GUI) for user interaction, enhancing the overall user experience.**

The solver's backtracking algorithm systematically fills the Sudoku grid, recursively checking for valid placements of numbers. The approach begins by selecting an empty cell and testing possible digits, backtracking when a conflict arises. This method ensures that all potential solutions are evaluated while minimizing redundant checks through intelligent heuristics, such as selecting the most constrained variable first. The efficiency of the algorithm is paramount, particularly for complex puzzles requiring numerous iterations.

Keywords- Backtracking, sudoku, cpp, Qt, GUI

I. INTRODUCTION

Sudoku, a widely recognized logic-based puzzle, has captivated enthusiasts across the globe for decades. The objective is straightforward: fill a 9x9 grid with digits from 1 to 9, ensuring that each row, column, and 3x3 sub-grid contains all digits without repetition. While the rules are simple, the challenge lies in solving it efficiently, especially for complex puzzles with minimal clues. This project addresses the inherent difficulty of the Sudoku problem by implementing a robust solver that automates the solution process through algorithmic techniques, significantly reducing the time and effort involved for users.

II. SOLUTION DESIGN

The proposed solution for the Sudoku solver is designed to utilize a backtracking algorithm, which is a form of depth-first search (DFS) specifically tailored for constraint satisfaction problems. The system is structured to include the core solver logic, user input handling, and a graphical user interface (GUI) for ease of interaction. The overall architecture consists of several components:

- Sudoku Grid Representation: The Sudoku puzzle will be represented as a 2D array (or matrix) of integers, where each cell can either hold a number between 1 and 9 or be empty (indicating unfilled). This representation allows for straightforward access and modification of individual cell values.
- Backtracking Algorithm: The core logic of the Sudoku solver will implement a backtracking algorithm that recursively attempts to fill the grid. The algorithm will start from the first empty cell, attempting to place each number from 1 to 9, using the validator class to check for conflicts. If a number placement leads to a valid state, the algorithm proceeds recursively to the next empty cell. If a conflict arises, it backtracks and tries the next number.
- Graphical User Interface (GUI): The GUI will be developed using a framework such as Qt or similar, allowing users to interact with the solver intuitively. The interface will include input fields for the initial Sudoku configuration, buttons to start the solving process, and visual cues to indicate the current state of the puzzle during execution.

III. ALOGRITHMIC STRATEGIES

- Backtracking Algorithm:

The primary strategy employed for solving Sudoku is backtracking, a depth-first search algorithm where numbers are tentatively placed in cells, and the algorithm recursively attempts to fill the grid. If a number leads to an unsolvable condition, the algorithm backtracks to try the next number.

This strategy is justified since it explores all possible configurations and guarantees a solution for valid puzzles within polynomial time.

- Random Generation and Validation:

When creating puzzles, the algorithm first generates a completely solved Sudoku grid, then randomly removes

numbers while ensuring a unique solution using recursive checking, which is efficient for generating valid puzzles.

IV. Time complexity analysis

1. Time Complexity:

The worst-case time complexity for the backtracking algorithm is O(9^n^2), where n is the number of empty cells (in this case, n=9). However, in practical scenarios, this is significantly reduced through early exit conditions and validating placements, leading to much quicker solving times.

2. Space Complexity:

The space complexity primarily results from the recursive stack for backtracking, which can go up to O(n^2) for the grid size. However, since only a constant amount of extra space is required for each cell placement, it remains manageable.

V. Implementation details

The Sudoku solver project represents a comprehensive approach to solving complex puzzle problems through an intelligently designed software architecture. At its core, the implementation leverages a recursive backtracking algorithm, which serves as the primary problem-solving mechanism. The project is structured around two primary classes: SudokuSolver, which encapsulates the core algorithmic logic, and MainWindow, which manages user interactions and visualization.

The software design emphasizes strict modularity and separation of concerns. The SudokuSolver class is meticulously crafted to handle distinct responsibilities, including puzzle validation, solving, and generation. By utilizing static methods, the implementation ensures efficient memory management and prevents unnecessary object instantiation. Each method is designed with a single, well-defined purpose - for instance, isSafe() systematically checks number placement validity across rows, columns, and 3x3 sub-grids, while solveSudoku() manages the recursive solving process.

Readability and maintainability were paramount in the code's design. The implementation adopts consistent naming conventions, using descriptive camelCase method names that clearly communicate their functionality. Type definitions, such as using SudokuGrid = std::vector<std::vector<int>>, provide flexibility in data structure representation while maintaining clean, abstracted code. The use of modern C++ features and standard library components demonstrates a commitment to writing idiomatic, efficient code that is both easy to understand and modify.

The heart of the Sudoku solver is its recursive backtracking algorithm, which employs a depth-first search strategy to solve puzzles. The solveSudoku() method recursively explores possible number placements, systematically backtracking when an invalid configuration is detected. Performance optimizations are seamlessly integrated into this approach. The isSafe() method acts as an early constraint satisfaction mechanism, dramatically reducing unnecessary recursive calls by preemptively validating number placements.

VI. Results and discussion

The Sudoku solver successfully demonstrated its effectiveness by accurately solving a wide variety of Sudoku puzzles while adhering to the core rules of the game. During testing, the implementation consistently verified that all solutions satisfied the fundamental constraints, ensuring that each number from 1 to 9 appeared only once in every row, column, and 3x3 sub-grid. This correctness was validated through rigorous checks applied during the solving process.

The algorithm efficiently handled multiple special and edge cases, including scenarios where puzzles had minimal initial numbers or required unique solutions. By incorporating explicit mechanisms for recognizing unsolvable configurations, the solver was able to prevent infinite recursion by backtracking as soon as a violation of Sudoku rules was detected. This capability ensured that the solver remained robust and efficient, even when encountering challenging scenarios.

In comparison to alternative algorithms, such as brute force techniques and constraint propagation methods, the backtracking approach used in this project strikes a favorable balance between simplicity and efficiency. While brute force methods can be straightforward to implement, they tend to be less efficient due to their exhaustive nature. On the other hand, constraint propagation methods require more complex implementations but can significantly reduce search space. The current backtracking algorithm minimizes unnecessary computations through strategic pruning, making it an ideal choice for a Sudoku solver.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Dr. Krishnappa H K, our project guide and faculty mentor, for his constant support, guidance, and encouragement throughout the development of this project. His expertise and insights greatly enhanced our understanding and approach toward solving algorithmic problems.

We are also thankful to the Department of Computer Science and Engineering, RV College of Engineering, for providing the necessary resources and a conducive environment to carry out this project successfully.

Lastly, we extend our appreciation to our peers and friends for their valuable feedback and continuous motivation, which played a vital role in shaping this work.

REFERENCES

[1] Norvig, P. (2006). Solving Every Sudoku Puzzle. [Online]. Available: http://norvig.com/sudoku.html

[2] Qt Documentation. [Online]. Available: https://doc.qt.io

[3] Levitin, A. (2011). Introduction to the Design and Analysis of Algorithms. Pearson.

[4] Knuth, D.E. (2000). The Art of Computer Programming: Volume 4. Addison-Wesley.